

Práctica 7: Análisis de Recorridos en Grafos y Optimización mediante Árboles de Expansión Mínima

Asignatura: Análisis de Algoritmos

Autores:

**Ariel Hernández Velázquez
Gael Neftalí Orduña Hernández**

Fecha de entrega:

12 de mayo de 2026

Universidad Nacional Autónoma de México
Facultad de Estudios Superiores Aragón

Índice

1. Introducción	2
2. Marco Teórico	2
2.1. Representación: Lista de Adyacencia	2
2.2. Estrategias de Recorrido	2
2.3. Optimización de Árbol de Expansión Mínima	2
3. Diseño de Implementación	2
4. Análisis de Complejidad	3
5. Pruebas y Análisis de Resultados	3
6. Conclusiones	3
7. Declaración de uso ético de IA	4

1. Introducción

El estudio de los grafos constituye uno de los pilares de la computación moderna, permitiendo modelar desde topologías de red hasta relaciones biológicas complejas. En esta práctica evaluable, se aborda el problema de la exploración de grafos mediante los algoritmos *Breadth-First Search* (BFS) y *Depth-First Search* (DFS). Asimismo, se profundiza en la optimización de recursos a través de la construcción de Árboles de Expansión Mínima (MST) empleando los algoritmos de Prim y Kruskal. El objetivo es no solo implementar estas soluciones, sino analizar su comportamiento bajo distintas densidades de aristas y tamaños de entrada.

2. Marco Teórico

2.1. Representación: Lista de Adyacencia

La eficiencia de un algoritmo de grafos depende críticamente de su estructura de datos subyacente. Se ha implementado una **lista de adyacencia** mediante diccionarios de Python. Esta elección ofrece una complejidad espacial de $O(V + E)$, superando a la matriz de adyacencia en grafos dispersos, donde $E \ll V^2$.

2.2. Estrategias de Recorrido

BFS (Breadth-First Search): Utiliza una estructura de cola (FIFO) para explorar los nodos de forma radial. Su propiedad fundamental es que en grafos no ponderados, la primera vez que se alcanza un nodo, se ha encontrado el camino más corto en términos de aristas.

DFS (Depth-First Search): Basado en una pila o recursión, explora cada rama hasta su profundidad máxima. Es vital para el análisis de conectividad y la identificación de ciclos mediante la clasificación de aristas en "aristas de árbol" y "aristas de retroceso".

2.3. Optimización de Árbol de Expansión Mínima

Un MST de un grafo $G = (V, E)$ es un subconjunto de aristas $T \subseteq E$ que conecta todos los vértices sin ciclos y con el peso mínimo total.

- **Prim:** Algoritmo ávido que construye el MST desde un vértice fuente, seleccionando siempre la arista de menor peso que conecta un nodo visitado con uno no visitado.
- **Kruskal:** Enfoque centrado en aristas. Requiere el uso de una estructura **Disjoint Set Union (DSU)** con *Path Compression* y *Union by Rank* para mantener la complejidad casi constante.

3. Diseño de Implementación

La arquitectura de la solución se divide en tres capas lógicas:

1. **Capa de Datos (grafo.py):** Implementación de la clase **Grafo** con métodos para manipulación dinámica de vértices y aristas.
2. **Capa de Algoritmos (recorridos.py y mst.py):**
 - Uso de `collections.deque` para eficiencia en BFS.
 - Implementación de `heapq` (Priority Queue) para optimizar la selección de aristas en Prim a $O(\log V)$.
 - Clase `UnionFind` para la gestión de componentes conexas en Kruskal.
3. **Capa Experimental (experimentos.py):** Automatización de pruebas de estrés y generación de métricas de rendimiento.

4. Análisis de Complejidad

A continuación se presenta la síntesis de las complejidades temporales y espaciales analizadas:

Algoritmo	Peor Caso (Tiempo)	Espacio	Estructura Clave
BFS / DFS	$O(V + E)$	$O(V)$	Cola / Pila recursiva
Prim (Heap)	$O(E \log V)$	$O(V + E)$	Priority Queue
Kruskal	$O(E \log E)$	$O(V)$	Union-Find

Cuadro 1: Comparativa técnica de algoritmos implementados.

Nota: En Kruskal, dado que $E \leq V^2$, $\log E$ es proporcional a $\log V$, por lo que ambas implementaciones de MST presentan rendimientos asintóticos similares.

5. Pruebas y Análisis de Resultados

Durante las pruebas unitarias, se verificó:

- **Detección de Ciclos:** El algoritmo DFS identificó correctamente ciclos en grafos cerrados y retornó *False* en estructuras de árbol lineales.
- **Consistencia de MST:** En el grafo K4 y el grafo de 5 vértices, tanto Prim como Kruskal convergieron al mismo peso total mínimo.
- **Rendimiento:** En grafos densos ($|E| \approx |V|^2/4$), Prim mostró una ligera ventaja sobre Kruskal debido a que evita el proceso inicial de ordenamiento global de aristas.

6. Conclusiones

La práctica permitió consolidar el conocimiento sobre estructuras de datos avanzadas. Se concluye que no existe un "mejor" algoritmo universal; mientras que Prim es eficiente cuando se tiene un grafo muy conectado, Kruskal destaca por su simplicidad y eficiencia en grafos dispersos. La implementación de *Union-Find* fue la pieza más desafiante, demostrando cómo una optimización matemática (rango y compresión) puede transformar radicalmente la escalabilidad de un algoritmo.

7. Declaración de uso ético de IA

Los autores declaran que se utilizó la inteligencia artificial **Gemini** como herramienta de apoyo para:

- Generar la plantilla base del documento en LaTeX.
- Verificar la sintaxis de las estructuras de datos en Python.
- Consultar definiciones teóricas para la redacción del marco teórico.

Todo el código ha sido revisado, probado y comprendido por los autores, garantizando que el aprendizaje ha sido el eje central de la actividad.

Referencias

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
- [2] Python Software Foundation. *heapq — Heap queue algorithm*. Disponible en docs.python.org.
- [3] Sedgewick, R., & Wayne, K. (2011). *Algorithms (4th Ed.)*. Addison-Wesley.